WHEN IS MODEL SOUPING TASTY? SIMILARITY, TRANSITIVITY, AND ROBUSTNESS

Simon Ghyselincks[1,2,*], **Pierre Mackenzie**[1,*], **& Evan Shelhamer**[1,2]
[1]Department of Computer Science, University of British Columbia
[2]Vector Institute
{sghyseli, pierrerl}@cs.ubc.ca

ABSTRACT

Model souping is a post-training technique where the parameters of models are averaged, often leading to improved performance over constituent models without increasing inference cost. However, the specific conditions required for success are not well understood, particularly regarding the trade-off between model diversity and stability. We analyse over 5,000 two-model ResNet-50 soups trained on CIFAR-100, with diversity controlled by branching ingredients from a shared training trajectory at varying epochs. We find that effective souping requires a balance: models must be similar enough to avoid model collapse, but diverse enough to yield improvements. We show that we can predict soupability relatively well with standard similarity metrics. Furthermore, we provide empirical evidence for the hypothesis that souping works by averaging within a low-loss basin by showing that souping is moderately transitive. We also observe that soup gains on corrupted data are strongly correlated with those on in-distribution data. Our findings yield practical advice for machine learning practitioners: if you want a tasty soup, use the right cooking time! Code and experiments are available at: https://github.com/chipnbits/too-salty.

1 INTRODUCTION

Wortsman et al. (2022) introduce *souping*: averaging model parameters produced by different fine-tuning trajectories from a common pre-trained model. In contrast to ensembling, which combines the *outputs* of models, souping combines the *parameters*, maintaining the same computational cost of inference with a single forward pass. Souping can yield better generalization than any one ingredient. Souping is formulated as a linear interpolation of model *parameters* θ using *weights* α; for the case of two models, $\theta_{\text{soup}} = (1-\alpha)\theta_A + \alpha\theta_B$. This not only captures the benefit of multiple optimization paths but also introduces an additional adaptation opportunity, as Croce et al. (2023) suggest that adjusting the interpolation weights enables intermediate behaviours that can better deal with distributional shifts.

Wortsman et al. (2022) hypothesise that souping works because fine-tuned models often lie within the same low-loss basin. A convex combination of their parameters is expected to remain within this basin while reducing the variance introduced by noisy training. They find that when the angle formed by the pre-trained model and ingredients to be souped together is larger, implying greater diversity, there is a greater performance boost from souping. Understanding why averaging maintains a low loss, how diversity contributes to robustness, and when souping is beneficial is therefore essential for studying adaptation more broadly.

Related Work: Souping has been used in a variety of settings. Croce et al. (2023) soup models trained to be robust to different distribution shifts and Ramé et al. (2023) soup ingredients trained on different tasks. Both cases lead to better generalisation. Jang et al. (2025) show that fine-tuned parameters are distributed around a low-loss center and use the angle between as few as two fine-tuned weights and their shared branch point to approximate this low-loss center.

[*]Equal contribution.

Souping is related to the idea of SWA Izmailov et al. (2019). In SWA, the ingredients of the soup come from different steps along the same training trajectory. By contrast, souping averages models from independent training trajectories from a shared initialization.

Souping success requires models to be 'compatible' in the sense that averaging their weights has low loss. This notion is closely related to 'stability to SGD noise' Frankle et al. (2020), enabled by linear mode connectivity between models trained from the same initialisation under different SGD noise. Such stability is only present after sufficient shared training Iyer et al. (2024). We propose that this defines window during training in which souping is effective and that pre-emptive souping leads to a high loss barrier, or 'model collapse'.

Our Contributions: Following these works, we seek to better understand souping by conducting a series of experiments addressing the following questions:

1. How much shared training is required for souping to be effective? We investigate varying the number of shared pre-training epochs before splitting into fine-tuned variants to empirically map the transition from model collapse to effective souping.
2. Can model similarity predict the effectiveness of souping? Identical models yield no generalization benefit, while overly dissimilar models fail to inhabit a shared low-loss region. We investigate whether standard similarity metrics can predict this balance between model compatibility and diversity.
3. Is souping transitive? If model A soups with B, and B soups with C, will A soup with C?
4. Does souping in-distribution predict souping out-of-distribution? While souping has been shown to help with robustness to distribution shifts, we seek to answer how correlated the soup gains are between in-distribution and out-of-distribution data.

More experiments on the effect of permuting models Ainsworth et al. (2023) prior to souping and how souping affects robustness can be found in Appendix A.1 and Figure A18 respectively.

Soups and Soup Gain: We soup pairs of models using the simple arithmetic mean $\theta_{\text{soup}} = \frac{1}{2}\theta_A + \frac{1}{2}\theta_B$. As shown by Ainsworth et al. (2023) and Iyer et al. (2024), the loss barrier typically has the most extreme behaviour at the midpoint, making it a useful summary statistic of souping performance across weights interpolation, accurately identifying souping failure. Characteristic weighting plots are shown in A19.

Soup gain is the test set accuracy gain of souping two models relative to the *best* parent model.

$$\text{soup gain} = \text{acc}(\theta_{\text{soup}}) - \max\{\text{acc}(\theta_A), \text{acc}(\theta_B)\}$$

where $\text{acc}(.)$ denotes test accuracy. We compare against the maximum accuracy of the ingredients, rather than the mean, as the purpose of a soup should be to improve over its ingredients. Soup gain can alternatively be measured as the improvement in loss.

Soup gain may lose information about the accuracy that the soup and the parents obtain. We investigate the relationship between soup gain and the mean parent accuracy and compare the soup gain/accuracy cumulative distribution functions (CDF) in Figures A3 and A5. We find that we lose very little information by looking at only the soup gain and proceed with this as our primary metric. We refer to soups with positive and negative soup gain as positive and negative soups.

2 Experiments

We train a baseline model for image classification on the CIFAR-100 dataset with ResNet-50 He et al. (2016) following Dadalto (2023), saving checkpoints every 10 epochs. From each checkpoint we train 4 new models with different optimizer settings, for details see Appendix Tables 2 and 3. All models are trained to convergence, with the best validation scored model saved for experiments, as shown in Figure A2. A total of 4 variants and 26 branch points were trained, yielding 104 related models and $5,356$ binary souping combinations for analysis. Additionally, we train 12 baseline models with and without Stochastic Weight Averaging (SWA) to compare the depth of fine-tuning paths against the breadth of SWA, see Appendix A.4.

1. Soup Gain and Shared Epochs: Figure 1 shows the CDF of soup gains.Many soups have extreme behaviour, with 40% of soups losing over 60% and only 14% with positive gain. There is also a sharp

transition to collapse, with only 20% soups between -70% and -20% gain. Grouping by the shared number of training epochs before branching (e.g., ingredients branched at epochs 50 and 100 share 50 epochs), we find the probability of positive soup gain generally increases with shared epochs, reaching around 80% after 280 shared epochs, see Figure 2. At this stage, no soups drop accuracy by more than 5%. However, peak gains ($> 0.5\%$) require a balance; if shared training is too extensive ingredients are not diverse enough, performance diminishes. Further analysis on mean gain, loss metrics, and model collapse is provided in Figures A7, A6 and A8.

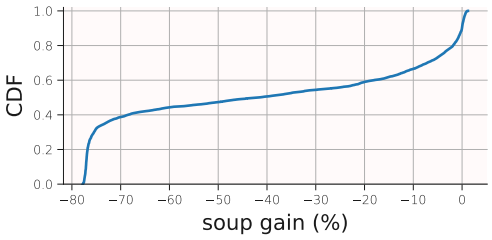

Figure 1: Empirical CDF of soup gain over all soups.

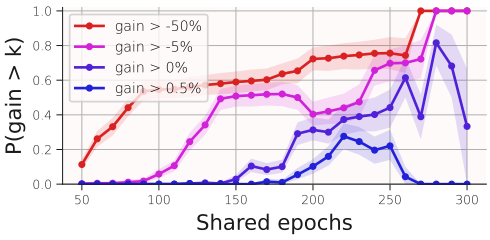

Figure 2: Probability of soup gain being greater than $k\%$ for varying k with 95% CIs.

2. Predicting Soupability with Similarity: To test if soupability is predictable, we compute a variety of similarity and distance metrics between model pairs. All metrics perform similarly, see Figure A10. Using KL divergence between the outputs of the ingredients as an example, we find a strong negative correlation with soup gain (Spearman -0.88), indicating divergence often leads to model collapse, see Figure 3. By contrast, gain among only positive soups has a positive correlation with dissimilarity (Spearman 0.16) and the most positive soups have some diversity, see Figure 4. Effective souping requires models to be sufficiently similar, but also rewards variety. Balancing these two effects is key to tasty soups. Additionally, shared epochs correlate with all similarity metrics, see Figure A9.

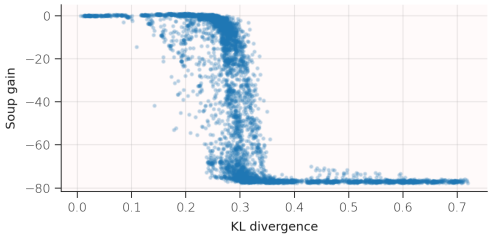

Figure 3: KL vs soup gain (Spearman -0.88).

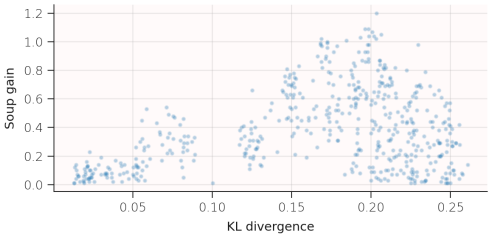

Figure 4: KL vs $(+)$ soup gain (Spearman 0.16).

3. Is Souping Transitive?: To test the hypothesis that soupable models lie in the same low-loss basin, we evaluate the transitivity of model triplets (A, B, C). Figure 5 shows that B and C are likely to soup only if A soups with both B and C. We observe a moderate positive correlation (Spearman 0.64) between the gain of (B, C) and the minimum gain of (A, B) and (A, C), see Figure A12.

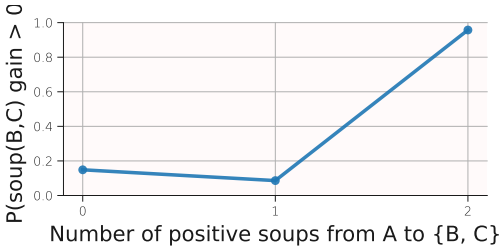

Figure 5: Probability of positive soup gain of B and C vs positive soups with A.

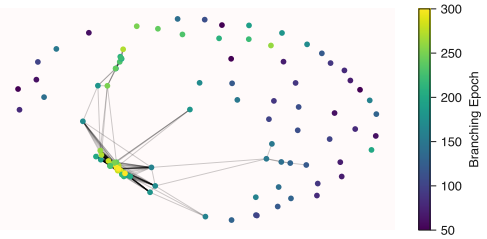

Figure 6: 2D embedding of 104 models using soup-gain distance; edges indicate $(+)$ soups.

We embed our 104 models into 2D using soup gain as a distance metric to search for souping clusters, plotting them in Figure 6, with details found in Section A.6. We find that all successful soups form a single connected component, and there is a dense cluster of models with higher shared epochs. Cases where transitivity fails exhibit a smooth but significant loss barrier, showing that the loss basin is non-convex with modes of interconnectivity, see Figure A13. We also find that such failures never exhibits model collapse, see Figure A14. We attempt to explain when transitivity fails in terms of the shared epochs and find further evidence of the sufficient-diversity tradeoff, see Tables 4 and 5.

4. Souping for Robustness to Corruption: To establish whether souping for in-distribution (ID) performance also increases out-of-distribution (OOD) performance, we compute the soup gain on CIFAR-100C (Hendrycks & Dietterich, 2019) with severity level 3. The soup gains on test and corrupted data correlate strongly (Spearman 0.99), even when restricted to positive soups (Pearson 0.61), see Figures A15, A17. Thus, ID performance improvement transfers to unseen target distributions. We also plot the probability of positive soup gain on corrupted data vs shared epochs in Figure A16. The probability of positive gain increases with the number of shared epochs for both clean and corrupted data but the corrupted data always has a lower probability of positive gain.

How does souping compare to SWA? Model accuracy on both clean and corrupted data has been shown to improve with souping. Here we compare the improvements to SWA in Table 1. SWA offers a significant boost in robustness to corruption, always improves the baseline run, and performs better than the best soups. We hypothesize that this is because in this experimental setting, SWA has explored a wider breadth than the binary soups and has more closely converged to the low-loss center described by Jang et al. (2025).

Type	**P(Gain > 0)**	**Mean Gain (Acc %)**	**Mean Corrupted Gain (Acc %)**
Soups: 200 Epochs	$39.7\% \pm 8.4\%$	$0.37\% \pm 0.06\%$	$0.42\% \pm 0.12\%$
Soups: 250 Epochs	$44.7\% \pm 11.2\%$	$0.49\% \pm 0.07\%$	$0.53\% \pm 0.09\%$
SWA (12 Runs)	100%	$1.7\% \pm 0.3\%$	$4.3\% \pm 0.5\%$

Table 1: Comparison of souping robustness to SWA. Note that gain is the mean over only positive soup. Including all soups would result in negative performance, as there are many instances of model collapse. SWA performs notably better. We hypothesise SWA explores a wider region of the low-loss basin than a binary soup can.

3 Conclusion

We have tasted more soups to better understand souping. Ingredients must have sufficiently many shared epochs of training in order to be compatible, but not so many that the soup gain is minimal. Various similarity measures between models correlate similarly with soup gain. Similar ingredients are less likely to collapse when souped, but very similar ingredients yield smaller soup gains. Thus, the right balance must be struck for the most effective souping. When souping, we encourage practitioners to test a range of similarities of ingredients to ensure they are finding an optimal soup. Our experiments showing that souping is mostly transitive support the low-loss basin hypothesis. Finally, soup gains on in-distribution data are strongly correlated with those on corrupted data.

Limitations: We investigated ResNet-50 on CIFAR-100 to enable a comprehensive combinatorial analysis of over 5,000 binary soups that would be computationally prohibitive with large foundation models. Iyer et al. (2024) demonstrated that the linear mode connectivity is a fundamental optimization property, suggesting our findings would transfer to larger-scale fine-tuning settings. Additionally, we only consider pairwise souping using the arithmetic mean at the midpoint for similar reasons. Other methods of souping, such as learned soups, may yield different results.

Future Work: Future empirical work could conduct similar experiments in different settings, such as a variety of model architectures and datasets. Theory could be developed for souping in simpler settings like an overparameterized linear model or a shallow network. Theory could also be created to help predict the change in loss we expect from souping associated with noise reduction. Validating the performance of SWA against model stock estimates of the low-loss center could confirm if model souping remains a strong contender for test-time robustness under distribution shift.

REFERENCES

Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git re-basin: Merging models modulo permutation symmetries, 2023. URL https://arxiv.org/abs/2209.04836.

Francesco Croce, Sylvestre-Alvise Rebuffi, Evan Shelhamer, and Sven Gowal. Seasoning model soups for robustness to adversarial and natural distribution shifts, 2023. URL https://arxiv.org/abs/2302.10164.

Eduardo Dadalto. Resnet-50 model trained on cifar-100. https://huggingface.co/edadaltocg/resnet50_cifar100, 2023. Hugging Face Model Repository.

Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M. Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis, 2020. URL https://arxiv.org/abs/1912.05671.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. In *Proceedings of 2016 IEEE Conference on Computer Vision and Pattern Recognition*, CVPR '16, pp. 770–778, Las Vegas, NV, USA, June 2016. IEEE. doi: 10.1109/CVPR.2016.90. URL http://ieeexplore.ieee.org/document/7780459.

Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations, 2019. URL https://arxiv.org/abs/1903.12261.

Gaurav Iyer, Gintare Karolina Dziugaite, and David Rolnick. Linear weight interpolation leads to transient performance gains. *Transactions on Machine Learning Research*, 2024. URL https://openreview.net/forum?id=XGAdBXlFcj. Presented at HiLD (ICML 2024 Workshop).

Pavel Izmailov, Dmitrii Podoprikhin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization, 2019. URL https://arxiv.org/abs/1803.05407.

Dong-Hwan Jang, Sangdoo Yun, and Dongyoon Han. Model stock: All we need is just a few fine-tuned models, 2025. URL https://arxiv.org/abs/2403.19522.

Alexandre Ramé, Kartik Ahuja, Jianyu Zhang, Matthieu Cord, Léon Bottou, and David Lopez-Paz. Model ratatouille: Recycling diverse models for out-of-distribution generalization, 2023. URL https://arxiv.org/abs/2212.10445.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time, 2022. URL https://arxiv.org/abs/2203.05482.

A Appendix

A.1 Permutation Alignment for Souping

Following the work from Ainsworth et al. (2023), we investigate whether permuting the neurons of models prior to souping increases the effectiveness of souping. We use the `rebasin`[1] package to align pairs of models before souping. This package uses the 'matching weights' method which permutes the neurones by inspecting only the weights. This contrasts with 'activation matching' which requires forward passes through the network, and 'straight through estimators' which are even more computationally expensive. The authors find that matching weights performs similarly to activation matching while being computationally cheaper. Therefore, we only consider the matching weights method.

We align all $5,346$ pairs of models using `rebasin` and compute the soup gain after alignment. Prior to permutation, 14.25% of soups were positive, while after permutation, 14.32% of soups were positive. However, 7.6% of soups obtained a loss higher than 5 — worse than the loss of any soup prior to permutation. We plot the cumulative distribution function (CDF) of the difference in soup gain before and after permutation in Figure A1. This plot shows that while permuting can sometimes help, it does not do so consistently. Further, the median difference in soup gain is approximately zero, indicating that permuting does not have a significant effect on the effectiveness of souping in our experiments. There also remains a significant risk of severe degradation. Thus, we conclude that 'matching weights' permutation does not make a noticeable different to soupability in our setting. We align all $5,346$ pairs of models using `rebasin` and compute the soup gain after alignment. Prior to permutation, 11.03% of soups were positive, while after permutation, 11.09% of soups were positive. However, 7.6% of soups obtained a loss higher than 5 — worse than the loss of any soup prior to permutation. We plot the cumulative distribution function (CDF) of the difference in soup gain before and after permutation in Figure A1. This plot shows that while permuting can sometimes help, it does not do so consistently. Further, the median difference in soup gain is approximately zero, indicating that permuting does not have a significant effect on the effectiveness of souping in our experiments. There also remains a significant risk of severe degradation. Thus, we conclude that 'matching weights' permutation does not make a noticeable different to soupability in our setting.

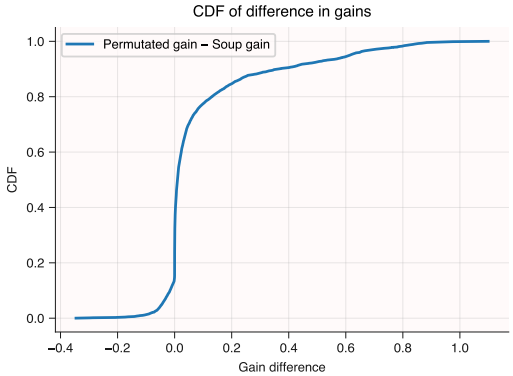

Figure A1: Cumulative distribution function (CDF) of the difference in soup gain before and after permutation alignment using `rebasin`. This ignores the 7% of soups with a loss higher than 5 after permutation as these make the plot difficult to interpret. The remaining mean and median difference is approximately zero, indicating that permutation alignment does not have a significant effect on the effectiveness of souping in our experiments. While some soups benefit from permutation alignment, others are negatively affected, leading to an overall negligible impact while there is a risk of severe degradation.

[1] `https://pypi.org/project/rebasin/`

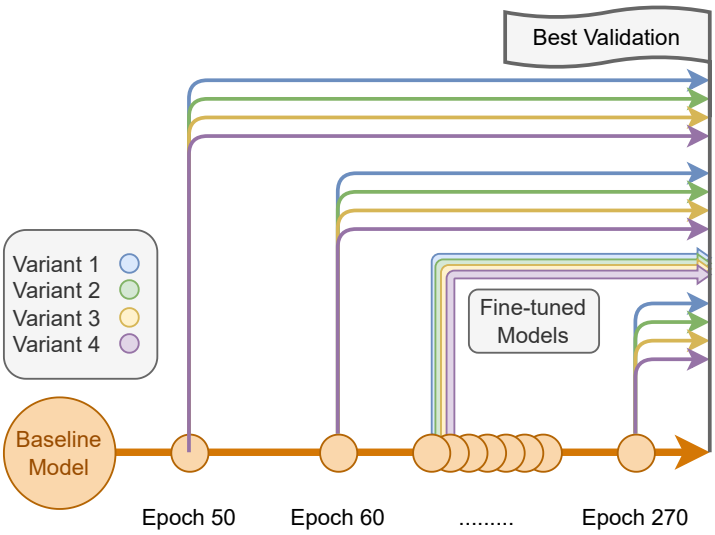

Figure A2: Branching of fine-tuned models from baseline checkpoints. A single baseline model is trained, with checkpoints saved every 10 epochs. From each checkpoint, 4 variants are trained with different optimizer hyper-parameter perturbations.

Model	Learning Rate Scale	Momentum Scale	Weight Decay Scale
Model 1	0.7073	1.1009	0.8284
Model 2	1.2244	0.9247	1.1150
Model 3	0.5112	1.0695	1.1099
Model 4	0.5373	0.8594	0.9078

Table 2: Optimizer perturbation scales applied during finetuning from the baseline ResNet-50 checkpoint on CIFAR-100. Each model scales the original SGD hyperparameters multiplicatively.

A.2 Training Details for CIFAR-100 with ResNet-50

Component	Hyperparameter	Value
Dataset	Dataset	CIFAR-100
	# Classes	100
	Data augmentation	Mirroring and Padded Offset
	Validation split	5% of training set
	Split seed	42
Model	Architecture	ResNet-50
	Pretrained	No (from scratch)
Optimization	Optimizer	SGD (Nesterov)
	Initial learning rate	0.1
	Momentum	0.9
	Weight decay	5×10^{-4}
Learning rate schedule	Scheduler	CosineAnnealingLR
	$T_{\max}$	300 epochs
	$\eta_{\min}$	0
Training	Epochs	300
	Batch size	128
	Mixed precision	No

Table 3: Training hyperparameters for CIFAR-100 with ResNet-50. A randomization seed of 42 is used unless otherwise specified.

A.3 Further Details on Experiments

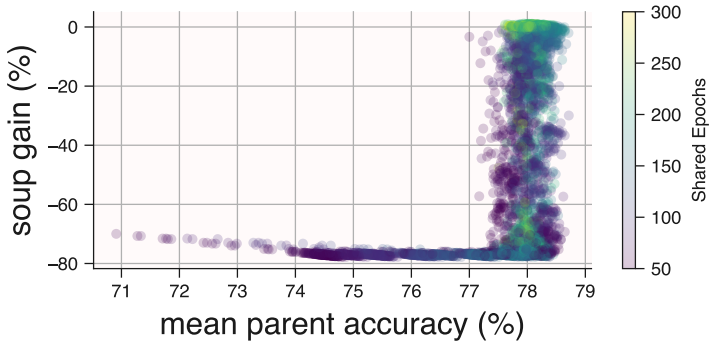

Figure A3: Mean accuracy of a soup's parents vs Soup gain. Color corresponds to the number of shared epochs between the soup's ingredients. We observe no clear trend, other than a tail of models which collapsed with the worst parents. We thus conclude we are not missing any hidden information based on the parents' performance and it is therefore reasonable to consider the soup gain as a metric in isolation.

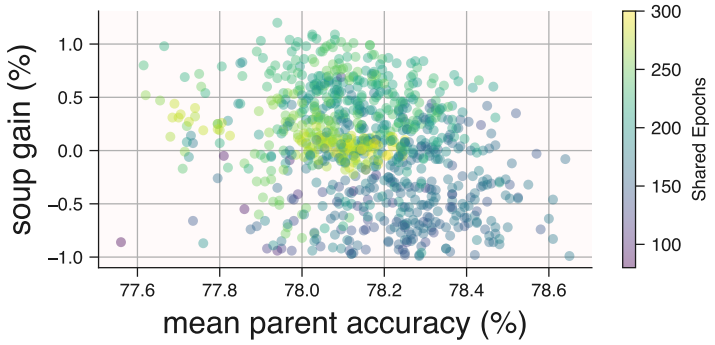

Figure A4: The same plot as in Figure A3 but restricted to only include soups with soup gain greater than -1% for clarity. Once again, we observe no clear pattern.

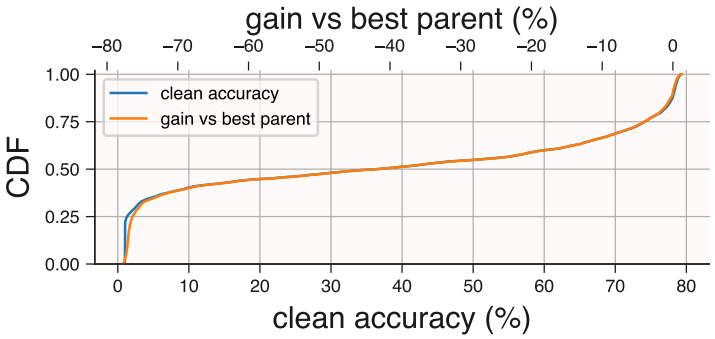

Figure A5: The empirical CDF of soup accuracy and soup gain over all our soups. We observe a variety of performances in an S-shape where there are few models with middling behaviour. The lines are also nearly indistinguishable. This implies that we can safely consider only soup gain without missing any effects contained in the soup accuracy.

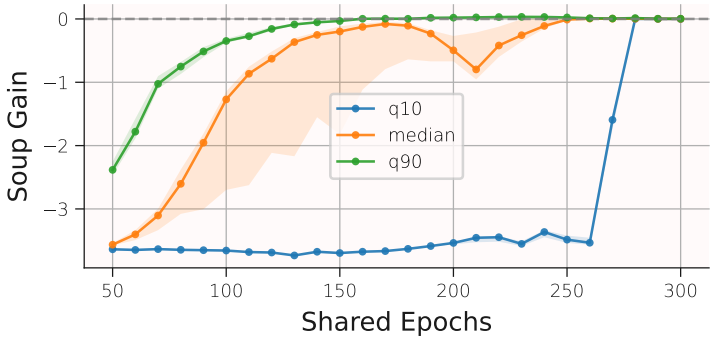

Figure A6: Quantiles of soup gain vs shared epochs with soup gain measured as the reduction in loss vs the best parent. Similar to the conclusions from Figure 2, we find model collapse almost never occurs past shared epochs 260.

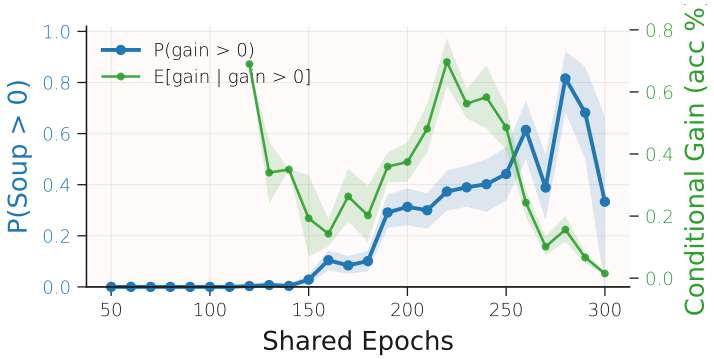

Figure A7: Probability of positive soup gain and conditional expected gain vs shared epochs. We find that the expected soup gain noisy, but is maximised in an ideal window of shared epochs. Past this point, and models are too similar, leading to minimal gains. Before this point, models are incompatible and rarely yield positive gain.

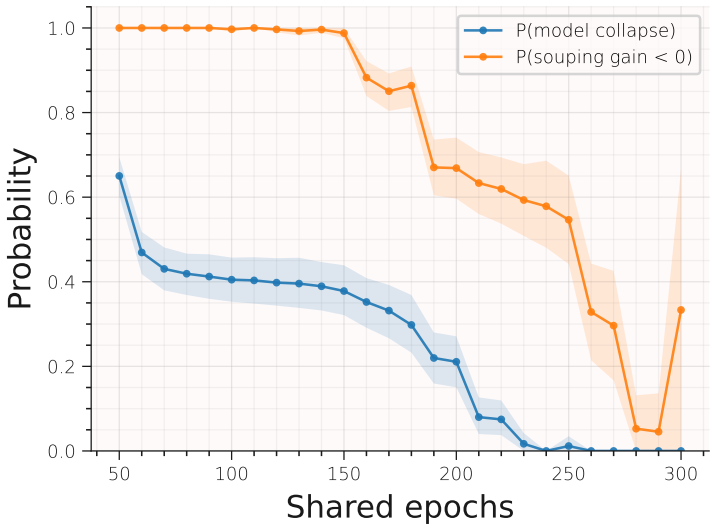

Figure A8: We plot both the probability of model collapse and the probability that soup gain is positive over varying shared epochs, with 95% bootstrapped confidence intervals. Model collapse is defined as the soup attaining less than 5% test accuracy. Both model collapse and negative soup gain decrease with shared epochs. In particular, model collapse is very rare past epoch 200 and never occurs past epoch 250.

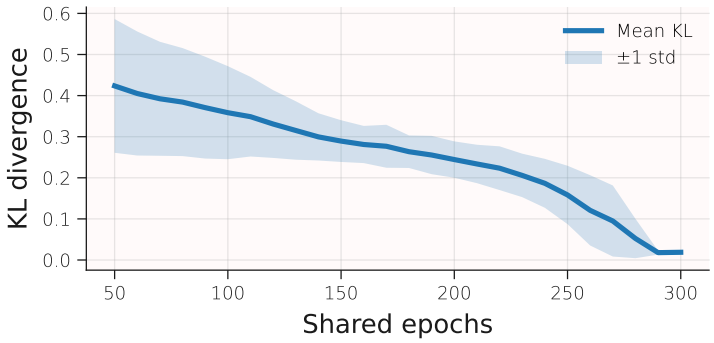

Figure A9: Shared epochs vs KL divergence. We plot the mean and standard deviation over each value of shared epochs. The full data has a Spearman correlation of -0.66. We conclude that we can noisily recover the number of shared epochs by measuring the KL divergence, and that model similarity can be controlled imprecisely by varying the number of shared epochs.

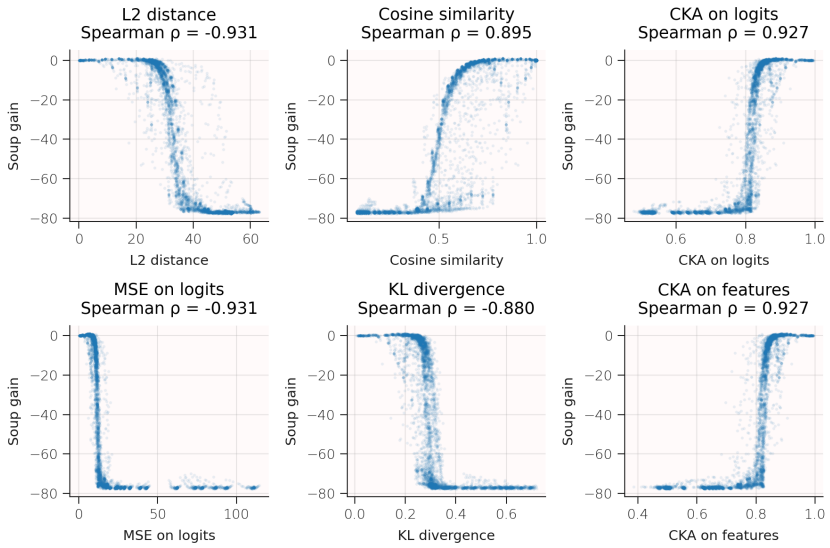

Figure A10: Soup gain vs various similarity and distance metrics between pairs of models. Each subplot shows the soup gain against one metric, with the Spearman correlation. All metrics perform similarly. We conclude that the more similar models are, the more likely souping is to not cause model collapse.

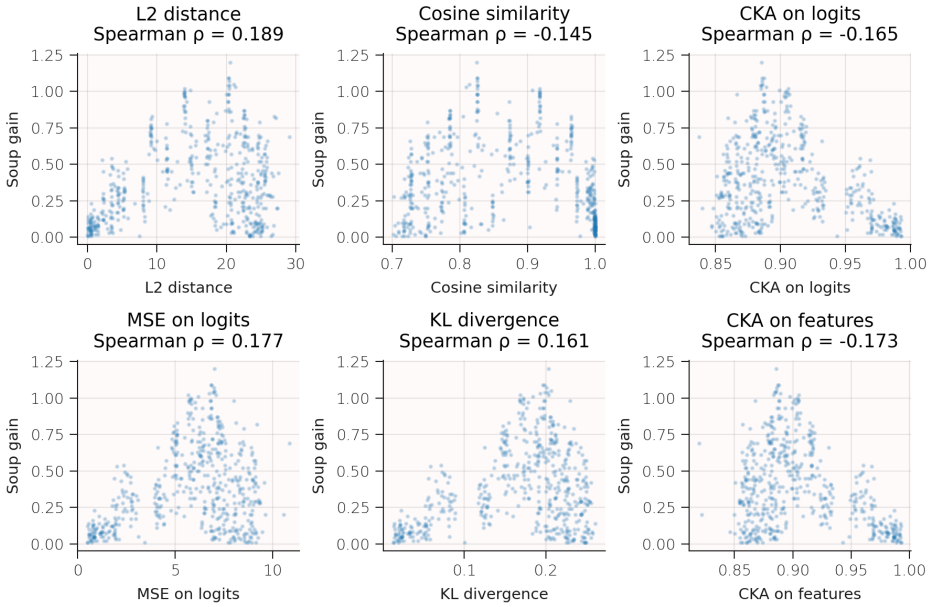

Figure A11: Soup gain vs various similarity and distance metrics between pairs of models, for the subset of soups with positive soup gain. Each subplot shows the soup gain against one metric, with the Spearman correlation. Each metric correlates similarly with soup gain. We see that when models are very similar, soup gain is small, while more dissimilar models can yield larger soup gains.

Table 4: Transitivity failure vs shared epochs over all (A, B, C) triples. Here n denotes the number of triples in the bin for which at least two soups are positive, and p_{fail} is the proportion of those triples for which the remaining soup is negative. Failure rates are highest for very small shared epochs, but no monotonic trend is observed. A sweet spot can be found in the 220–240 epoch bin, once again suggesting some diversity is beneficial.

Shared epochs (ABC)	n	p_{fail}
120–180	5742	0.214
190–190	2682	0.072
200–210	4386	0.056
220–240	4521	0.024
250–300	3507	0.042

Table 5: Transitivity failure vs branching-epoch difference $|B-C|$. Here n and p_{fail} are defined as in Table 4. We restrict to triples with shared epochs in $[220, 240]$, fix the lowest-epoch model, and vary the branching-epoch difference of the remaining two models. We observe that p_{fail} is lower with a greater value of $|B-C|$, suggesting that some diversity is beneficial, but the effect size is small.

Epoch diff (BC)	n	p_{fail}
0–10	534	0.062
20–20	306	0.049
30–40	393	0.025
50–70	164	0.024

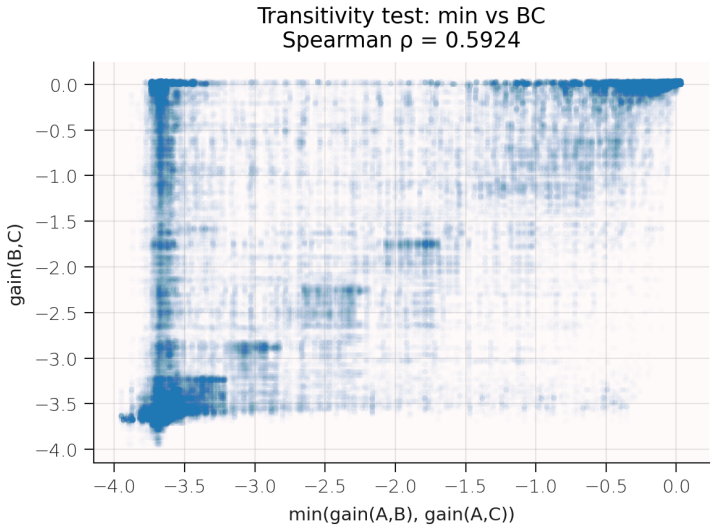

Figure A12: Scatterplot of the soup gain of models B and C against the minimum soup gain of models A with B and C. Each point represents a triplet of models (A, B, C). We observe a positive Spearman correlation of 0.59, suggesting that souping is fuzzily transitive. The correlation with the mean soup gain is lower, at 0.48.

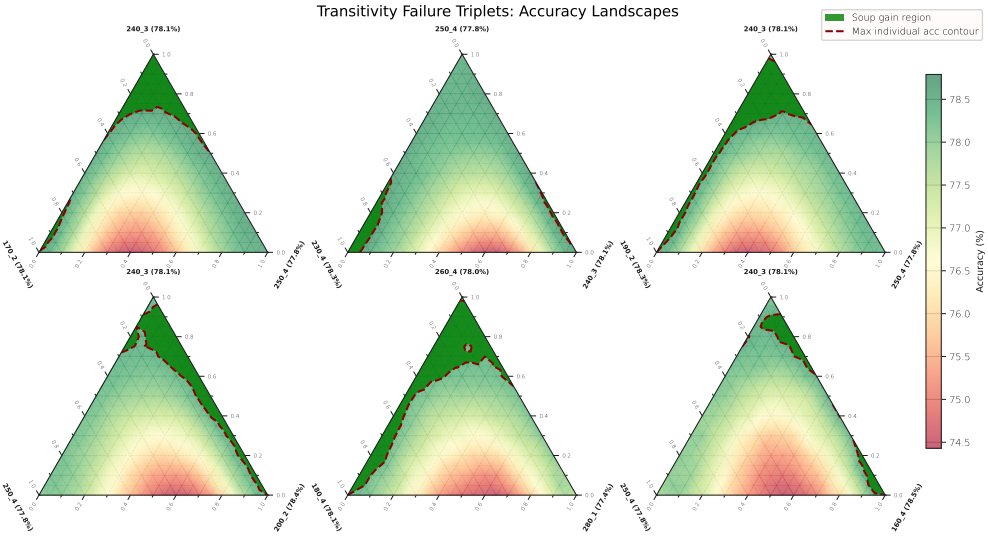

Figure A13: A fine-resolution raster image showing clean accuracy performance over a convex combination of model triplets that have failed the transitivity test. Note the left and right edges of the triangle are two-model soups that provide gain, while the third bottom edge shows mild souping failure, with a smooth loss barrier in red. The green area and dashed red boundary show clean accuracy that meets or exceeds all three constituent models.

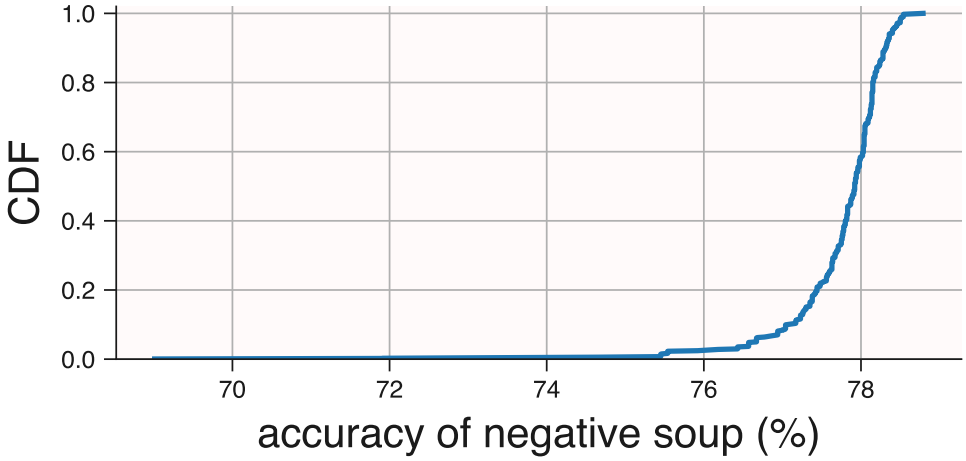

Figure A14: Empirical CDF of the accuracy of the negative soup in a triangle with 2 positive edges. There are no negative soups which exhibit severe model collapse, and most attain a reasonable performance relative to our best models (¿ 75%). We believe this is because with 2 positive edges, the loss basin is likely to be flatter than with 0 or 1 positive edge, and so the negative soup is unlikely to cause model collapse.

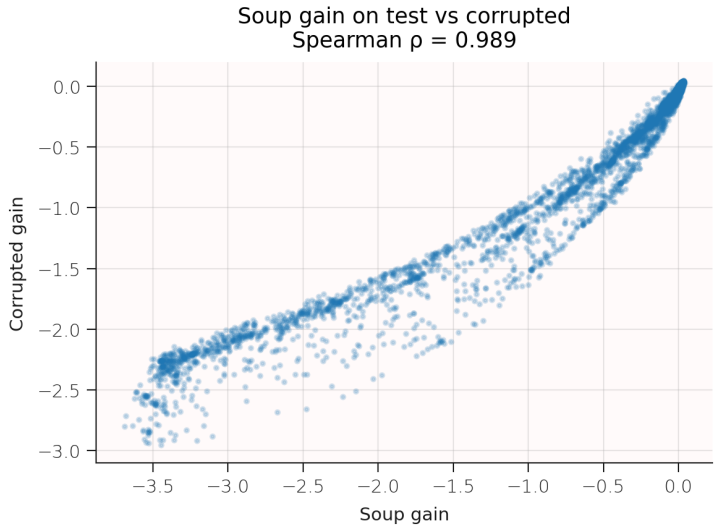

Figure A15: Scatterplot of the soup gain on test vs corrupted data. There is a clear sub-linear trend with strong correlation. Such a close relationship is sensitive to the nature of the distribution shift. This plot mostly shows that when model collapse occurs on the original test set, it also occurs on the corrupted data.

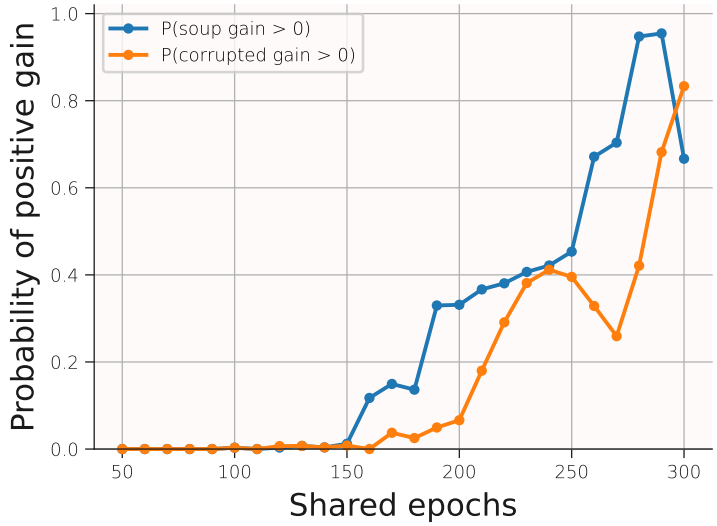

Figure A16: Probability of positive gain for soups as a function of the number of shared epochs. We see that the probability of positive gain increases with the number of shared epochs, for both clean and corrupted data. However,the corrupted data consistently has a slightly lower probability of positive gain. Thus, while souping also helps on corrupted data, it is slightly less effective than on clean data.

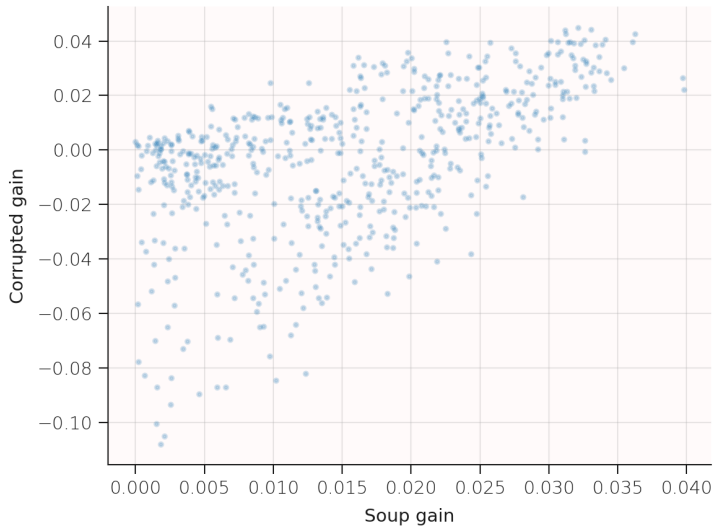

Figure A17: Plot of soup gain on test vs corrupted data for only models with positive soup gain on the test set. Spearman correlation of 0.56.

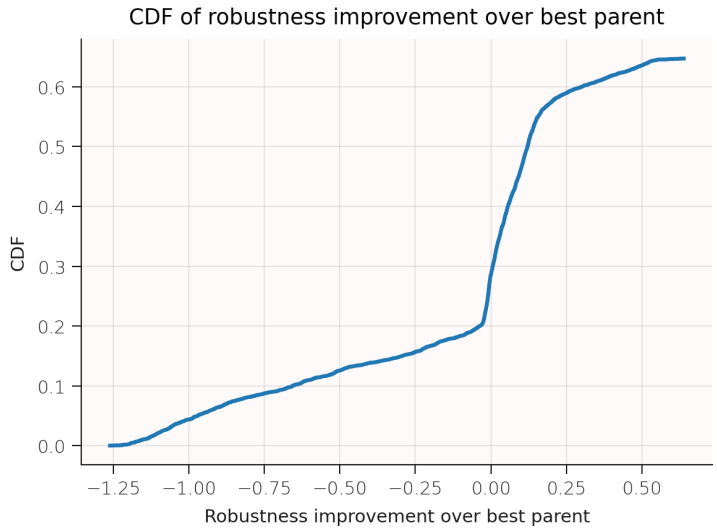

Figure A18: CDF of the difference in *robustness gap* before and after souping. The robustness gap is defined as the difference in loss between the test set and the corrupted set. The robustness gap before souping is taken as the minimum robustness gap of the parents. We see a fairly symmetric distribution. It has mean -0.04 and median 0.02. We therefore conclude that souping does not systematically improve the robustness gap.

A.4 SWA Experiment Runs

Baseline training was performed using the same hyper-parameters as in 3 and using the top validation model. We compare to SWA starting at epoch 220 from the same base model. The initialization and batch ordering seeds with full results are shown below in 6.

	Clean Data						**Corrupted Data**					
	Accuracy (↑)			Loss (↓)			Accuracy (↑)			Loss (↓)		
Seed	Base	SWA	Δ	Base	SWA	Δ	Base	SWA	Δ	Base	SWA	Δ
42	78.42	79.75	**+1.33**	1.02	0.89	**-0.13**	51.87	55.45	**+3.58**	2.32	2.44	+0.12
43	78.04	79.90	**+1.86**	1.04	0.84	**-0.20**	50.76	54.89	**+4.13**	2.38	2.36	**-0.02**
44	78.79	80.81	**+2.02**	1.00	0.86	**-0.14**	50.83	54.67	**+3.84**	2.38	2.56	+0.18
45	78.53	80.14	**+1.61**	1.02	0.87	**-0.15**	51.36	55.45	**+4.09**	2.34	2.41	+0.07
46	78.49	80.29	**+1.80**	1.01	0.86	**-0.14**	51.31	55.13	**+3.82**	2.34	2.49	+0.15
47	78.30	80.43	**+2.13**	1.02	0.88	**-0.14**	51.09	55.86	**+4.77**	2.37	2.52	+0.15
48	78.96	80.13	**+1.17**	1.00	0.89	**-0.11**	51.36	55.17	**+3.81**	2.36	2.58	+0.22
49	78.12	80.40	**+2.28**	1.03	0.86	**-0.17**	50.39	54.79	**+4.40**	2.38	2.42	+0.04
50	78.26	79.85	**+1.59**	1.03	0.87	**-0.16**	51.47	56.06	**+4.59**	2.33	2.31	**-0.02**
51	77.79	79.58	**+1.79**	1.05	0.90	**-0.15**	50.28	55.47	**+5.19**	2.39	2.41	+0.02
52	78.97	80.56	**+1.59**	0.99	0.83	**-0.15**	51.26	55.58	**+4.32**	2.35	2.37	+0.02
53	78.25	79.48	**+1.23**	1.03	0.91	**-0.11**	50.87	55.47	**+4.60**	2.36	2.42	+0.06
Mean	78.41	80.11	**+1.70**	1.02	0.87	**-0.15**	51.07	55.33	**+4.26**	2.36	2.44	+0.08
Std	0.36	0.41	0.35	0.02	0.02	0.02	0.46	0.42	0.47	0.02	0.08	0.08

Table 6: Comparison of basline and SWA models branching from Epoch 220.

A.5 Binary Model Soup Accuracy Curves

While our experiments make a statistical analysis on the characteristics of even-weighted binary soups, we verify the validity of our simplifying assumptions with characteristic accuracy curves across 16 binary pairs to support our claim that even-weighted soups are a good measure for performance. For soups exhibiting model collapse, the measure is much more accurate, while for positive gain soups the metric is indicative of soupability but not as accurate for actual souping gain.

A.6 Details for Model Embedding Creation

Given our evidence for transitivity, we consider the broader landscape of all 104 of our originally trained models. Do the soups all exist in separate clusters of models in separate loss basins? We define a distance metric defined as

$$d_{AB} = -\operatorname{sign}(\text{soup gain}) - 0.1 * \text{soup gain}$$

where d_{AB} is the distance between models A and B. Intuitively, this metric puts models close together that soup together positively, taking into account the magnitude of soup gain. We then cast this down into a 2-dimensional embedding using Multidimensional Scaling. We also color by branching epoch and mark edges that represent successful soups. The resulting plot is shown in Figure 6.

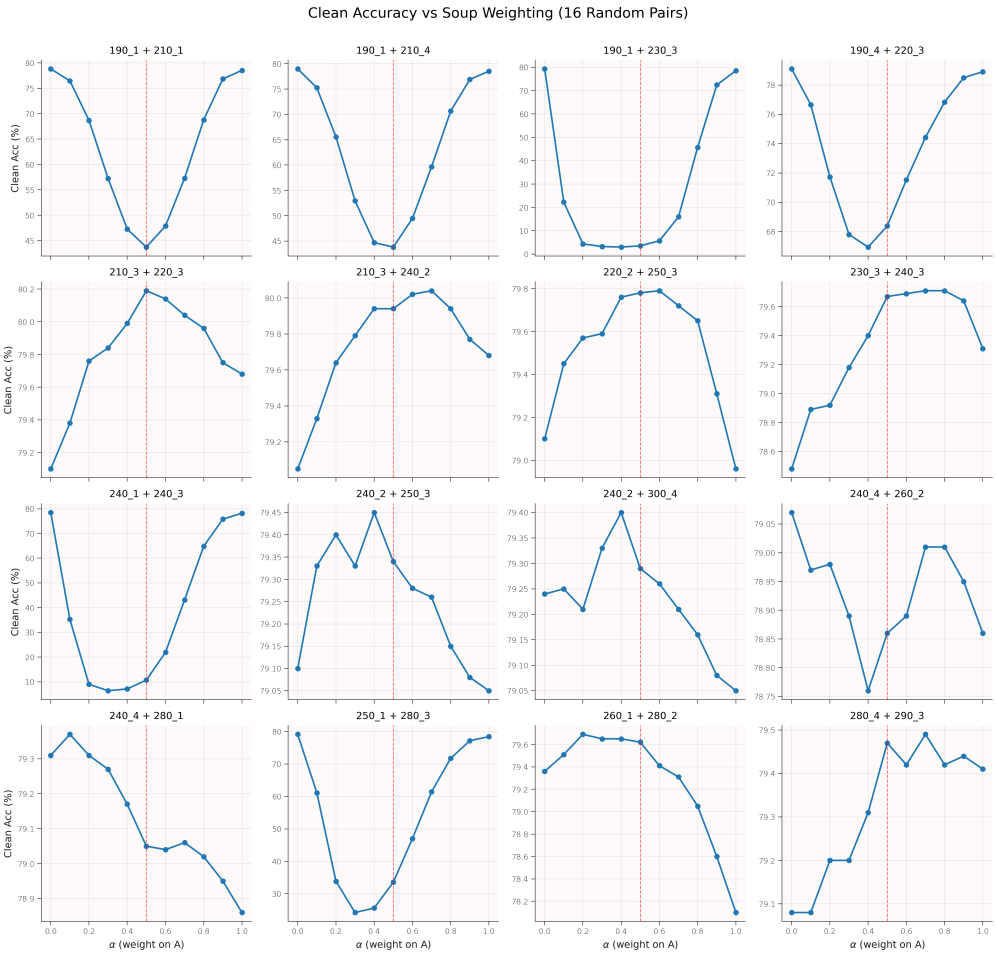

Figure A19: Souping characteristics across a range of parameter interpolation values for 16 models.